

Explication du code Python

Modèle de recommandations multi-sorties avec Keras/TensorFlow

Ce document explique, section par section, le fonctionnement du script fourni. L'objectif du code est d'entraîner un réseau de neurones capable de prédire, pour un « profil » donné (5 réponses à un questionnaire), le niveau de priorité (Absente, Moyenne, Elevée, Critique) de chacune des recommandations possibles.

1. Vue d'ensemble

Le script effectue quatre grandes étapes :

- 1 Construire la liste fixe de toutes les recommandations possibles.
- 2 Générer un jeu de données (tous les profils possibles + versions « bruitées »).
- 3 Construire un modèle de réseau de neurones à sorties multiples (une sortie par recommandation).
- 4 Entraîner le modèle, puis tester une prédiction sur un profil précis.

2. Les constantes de base

```
PRIORITES = ["Critique", "Elevée", "Moyenne"]  
CLASSES = ["Absente", "Moyenne", "Elevée", "Critique"]
```

CLASSES définit les 4 catégories possibles pour chaque recommandation, et surtout leur **ordre** : l'index 0 correspond à « Absente », l'index 1 à « Moyenne », etc. Cet ordre est important car le réseau de neurones prédit un *indice de classe*, pas directement un texte.

3. Figer la liste des recommandations

```
def lister_toutes_les_recommandations():  
    noms = []  
    vus = set()  
    for num_q in range(1, 6):  
        for recs in lesrecommandations[num_q]:  
            for r in recs:  
                if r["recommandation"] not in vus:  
                    vus.add(r["recommandation"])  
                    noms.append(r["recommandation"])  
    return noms  
  
LISTE_RECOS = lister_toutes_les_recommandations()  
NB_RECOS = len(LISTE_RECOS)  
INDEX_RECO = {nom: i for i, nom in enumerate(LISTE_RECOS)}
```

Le questionnaire comporte 5 questions (num_q de 1 à 5), et chaque réponse possible est associée à une liste de recommandations. Cette fonction parcourt tout et construit **une seule fois** la liste complète des recommandations distinctes (sans doublons), dans un ordre stable.

C'est essentiel : le réseau de neurones aura une « tête de sortie » par recommandation. Il faut donc que la sortie n° k corresponde toujours à la même recommandation, à l'entraînement comme à la prédiction.

INDEX_RECO permet de retrouver rapidement l'indice d'une recommandation à partir de son nom.

4. Construire la « cible » (labels) pour un profil

```
def construire_cible(profil, forcer_critique=False):
    cible = np.zeros(NB_RECOS, dtype=int) # 0 = Absente par défaut
    for num_q, num_r in enumerate(profil, start=1):
        recs = lesrecommandations[num_q][num_r - 1]
        for r in recs:
            idx = INDEX_RECO[r["recommandation"]]
            if forcer_critique:
                cible[idx] = CLASSES.index("Critique")
            else:
                cible[idx] = CLASSES.index(r["priorite"])
    return cible
```

Pour un **profil** donné (liste de 5 réponses, une par question), cette fonction construit un vecteur de taille **NB_RECOS** : pour chaque recommandation, elle indique quelle est sa priorité (0=Absente par défaut, sauf si le profil déclenche cette recommandation, auquel cas on met l'indice de la priorité correspondante).

Le paramètre **forcer_critique** sert à créer des exemples « bruités » où toutes les recommandations déclenchées sont forcées au niveau « Critique », quelle que soit leur priorité d'origine (voir section suivante).

5. Construction du jeu de données

```
def construire_dataset(seuil_bruit=30.0):
    X, Y = [], []
    for profil in itertools.product(range(1, 5), repeat=5):
        profil = list(profil)
        X.append(profil)
        Y.append(construire_cible(profil, forcer_critique=False))
        risque = score_risque_global(profil)
        if risque["pct_risque"] > seuil_bruit:
            X.append(profil)
            Y.append(construire_cible(profil, forcer_critique=True))
    return np.array(X, dtype=float), np.array(Y, dtype=int)
```

itertools.product(range(1, 5), repeat=5) génère **tous** les profils possibles : chaque question a 4 réponses possibles (1 à 4) et il y a 5 questions, donc **4⁵ = 1024 profils** au total.

Pour chaque profil, on ajoute un exemple « propre » (Y = la cible réelle selon les règles). Si le score de risque global du profil dépasse le seuil (30% par défaut), on ajoute **en plus** une version « bruitée » où tout est forcé à « Critique ». L'idée est d'apprendre au modèle que les profils à haut risque doivent être traités de façon plus sévère, en dupliquant ces cas dans le jeu d'entraînement.

Résultat : X est un tableau de forme (N, 5) — N profils, 5 réponses chacun — et Y est un tableau de forme (N, NB_RECOS) — pour chaque exemple, la classe de priorité de chaque recommandation.

6. Construction du modèle (réseau de neurones)

```
def construire_modele():
    entree = keras.Input(shape=(5,), name="profil")
```

```

x = keras.layers.Dense(32, activation="relu")(entree)
x = keras.layers.Dense(32, activation="relu")(x)

sorties = [
    keras.layers.Dense(4, activation="softmax", name=f"reco_{i}")(x)
    for i in range(NB_RECOS)
]

modele = keras.Model(inputs=entree, outputs=sorties)
modele.compile(
    optimizer="adam",
    loss=["sparse_categorical_crossentropy"] * NB_RECOS,
    metrics=[["accuracy"]] * NB_RECOS,
)
return modele

```

C'est un modèle **multi-sorties** (« multi-task learning ») avec un **tronc commun** et une **tête par recommandation** :

- **Entrée** : un vecteur de 5 nombres (les 5 réponses du profil).
- **Tronc commun** : deux couches denses de 32 neurones (ReLU), qui apprennent une représentation partagée du profil, utile pour toutes les recommandations.
- **NB_RECOS têtes de sortie** : une couche dense de 4 neurones avec activation *softmax* par recommandation. Chaque tête prédit une distribution de probabilité sur les 4 classes (Absente/Moyenne/Elevée/Critique) pour **une** recommandation précise.

Chaque sortie a sa propre fonction de perte (*sparse_categorical_crossentropy*, adaptée à une classification multi-classes avec labels entiers) et sa propre métrique (*accuracy*). Keras additionne les NB_RECOS pertes pour entraîner le modèle globalement.

7. Entraînement

```

Y_par_tete = [Y[:, i] for i in range(NB_RECOS)]

modele = construire_modele()
historique = modele.fit(
    X, Y_par_tete,
    epochs=15,
    batch_size=32,
    validation_split=0.2,
    verbose=2,
)

```

Comme le modèle a plusieurs sorties, Keras attend une **liste** de tableaux cibles (un par tête), d'où la découpe **Y_par_tete**. Le modèle est ensuite entraîné pendant 15 *epochs* (passages complets sur les données), avec 20% des données réservées à la validation pour surveiller le sur-apprentissage (*overfitting*).

8. Prédiction sur un profil de test

```

profil_test = [1, 3, 2, 4, 1]
pred = modele.predict(np.array([profil_test], dtype=float), verbose=0)

for i, nom in enumerate(LISTE_RECOS[:15]):
    classe_predite = CLASSES[np.argmax(pred[i][0])]

```

```
print(f" [{classe_predite:<9}] {nom[:60]}...")
```

Comme le modèle a plusieurs sorties, **pred** est une liste de NB_RECOS tableaux (un par tête). **pred[i]** correspond à la recommandation n°i, et **pred[i][0]** est le vecteur de 4 probabilités pour le premier (et seul, ici) exemple du batch.

np.argmax(pred[i][0]) renvoie l'indice de la classe la plus probable (0 à 3), que l'on convertit en texte via **CLASSES[...]**. La boucle n'affiche que les 15 premières recommandations de **LISTE_RECOS** (par souci d'exemple), pas les NB_RECOS au total.

Pourquoi certaines recommandations affichent « Absente »

Le modèle choisit **toujours** la classe la plus probable, même si sa confiance est faible. Si « Absente » apparaît souvent, les causes les plus courantes sont :

- **Déséquilibre des classes** : la majorité des couples (profil, recommandation) sont réellement « Absente » dans les données (une recommandation donnée n'est déclenchée que par certains profils), donc le modèle apprend que c'est souvent le choix le plus sûr.
- **Entraînement insuffisant** : seulement 15 epochs et un réseau assez petit (32 neurones) peuvent ne pas suffire pour bien séparer les classes rares.
- **Comportement attendu** : si une recommandation n'est effectivement pas déclenchée pour ce profil précis, « Absente » est la bonne réponse, pas un bug.

Pour vérifier lequel de ces cas s'applique, il est utile d'afficher le vecteur complet de probabilités par recommandation, plutôt que seulement la classe gagnante — cela révèle si le modèle hésite (probabilités proches) ou s'il est réellement confiant en « Absente ».

9. Schéma récapitulatif

Étape	Rôle
lister_toutes_les_recommandations()	Fixe la liste et l'ordre des NB_RECOS recommandations
construire_cible(profil)	Transforme un profil en vecteur de labels (1 par recommandation)
construire_dataset()	Génère les 1024 profils + versions bruitées (X, Y)
construire_modele()	Réseau à tronc commun + NB_RECOS têtes softmax (4 classes)
modele.fit(...)	Entraînement sur (X, Y) pendant 15 epochs
modele.predict(...)	Prédit, pour un profil donné, la priorité de chaque recommandation